TradePilot RAG — Implementation Guide

> This document summarises the architecture review findings and what needs to be built. Issues are ordered by priority. Each section explains **what the problem is, why it matters**, and **exactly what to do**.

Quick Overview

The current RAG pipeline works at small scale but has several gaps that will cause quality and reliability issues as the product grows. Here's a summary of what needs to change and roughly how long each thing takes:

Priority	Task	Effort
🔴 P0	Add vector index (halfvec + HNSW)	~2 hours
🔴 P0	Add JWT auth + rate limiting to chat function	~3 hours
🟠 P1	Add a reranker (Cohere)	~1 day
🟠 P1	Add hybrid search (BM25 + vector)	~1 day
🟠 P1	Add query rewriting for multi-turn conversations	~half day
🟠 P1	Switch to real token streaming, remove fake progress bar	~1 day
🟠 P1	Build a basic eval suite	~1 day
🟡 P2	Batch embeddings in ingest	~2 hours
🟡 P2	Replace validator with retrieval score checks	~2 hours
🟡 P2	Structured citation output (replace regex)	~half day

Key Decisions (Already Made — Don't Re-debate These)

Before diving in, four architectural forks were evaluated. These are settled. Implement the recommended option.

1. Embedding storage — `halfvec(3072)` ✅ Two options: change the column to `halfvec(3072)` (half-precision, same model, ~1% recall loss, half the storage) vs. switch to a 1024-dim embedding model entirely and re-embed everything (more work, possibly marginally better retrieval on some benchmarks). **Go with `halfvec(3072)`.** It's a single migration, no re-embedding, no vendor change, and the recall difference is negligible in practice.

2. Reranker vendor — Cohere rerank-3.5 ✅ Two options: Cohere `rerank-3.5` (paid API, ~$1/1k queries, best-in-class accuracy, 2 lines of code to add) vs. self-hosting BGE reranker on a Supabase edge function (free, but slower cold starts, more infrastructure to maintain, worse accuracy than Cohere 3.5). **Go with Cohere.** At any reasonable query volume the cost is trivial and the setup is an afternoon vs. a week.

3. Eval framework — Langfuse ✅ **(or Promptfoo if you want evals only)** Three options considered: Promptfoo (lightweight, YAML config, eval-only), Inspect (Anthropic's own eval framework, Python-based), Langfuse (tracing + evals in one place). **Go with Langfuse** — it covers both observability (traces, costs, latency per LLM call) and evals in the same dashboard, which means you're not maintaining two separate tools. If you genuinely only want evals and already have observability covered elsewhere, Promptfoo is fine and simpler to set up.

4. Streaming — rip and replace ✅ Two options: remove the progress animation entirely and stream Claude tokens directly, or keep the existing progress steps and layer token streaming on top. **Rip and replace.** The progress steps (generating, retrieving, validating…) are not worth the ~80 lines of state machine they require. Real token streaming is a better experience and less code. There's nothing in the progress steps worth preserving.

🔴 P0 — Fix the Vector Index

What's wrong

`tp_knowledge_chunks.embedding` has **no index at all**. Every chat query does a full table scan — it computes cosine distance against every row in the database. At 100 chunks you won't notice. At 10,000 chunks, queries take 1–3 seconds. At 100,000 chunks, the system effectively stops working.

There's also a trap: the column is `vector(3072)` (Gemini's output size), but pgvector's HNSW index only supports up to 2,000 dimensions. So you can't just add an index — you need to change the column type first.

What to do

Change the column to `halfvec(3072)` — this is a half-precision vector type that pgvector supports up to 4,000 dims. The recall difference is tiny (~1%) and storage halves. Then add the HNSW index.

```sql
```

```
-- Run this migration
ALTER TABLE tp_knowledge_chunks
  ALTER COLUMN embedding TYPE halfvec(3072) USING embedding::halfvec(3072);

CREATE INDEX tp_knowledge_chunks_embedding_idx
  ON tp_knowledge_chunks
  USING hnsw (embedding halfvec_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

Then update `tp_match_knowledge_chunks` to cast the query embedding to `halfvec` before the similarity search. That's the full change.

Do this before the dataset grows past a few thousand chunks.

🔴 P0 — Add Auth + Rate Limiting to the Chat Function

What's wrong

`useTradePilotChat.ts` calls the chat edge function using `VITE_SUPABASE_ANON_KEY`, which is **bundled in the browser and publicly visible**. The edge function has no auth check, no rate limit, and no per-user restriction. Each query triggers Gemini + Claude + optionally Serper. Anyone who finds the URL can send unlimited requests and run up the API bill.

Additionally, none of the `tp_*` tables have Row Level Security (RLS) enabled. Any authenticated user can currently read all chunks, all query logs, and all other users' question history.

What to do

Step 1 — Require a real JWT in the edge function

At the top of the `trade-pilot-chat` Deno function, verify the user's JWT before doing anything else:

typescript

```
const authHeader = req.headers.get("Authorization");
if (!authHeader) return new Response("Unauthorized", { status: 401 });

const { data: { user }, error } = await supabase.auth.getUser(
  authHeader.replace("Bearer ", "")
);
if (error || !user) return new Response("Unauthorized", { status: 401 });
```

Step 2 — Add a simple rate limit

Use `tp_query_logs` (which already exists) to count recent queries per user:

```typescript
const { count } = await supabase
  .from("tp_query_logs")
  .select("*", { count: "exact", head: true })
  .eq("user_id", user.id)
  .gte("created_at", new Date(Date.now() - 60 * 60 * 1000).toISOString()); // last h

if (count >= 30) return new Response("Rate limit exceeded", { status: 429 });
```

Step 3 — Enable RLS on all `tp_*` tables

```sql
-- Enable RLS (deny all client access by default — the safe starting state)
ALTER TABLE tp_knowledge_chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE tp_query_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE tp_knowledge_sources ENABLE ROW LEVEL SECURITY;
ALTER TABLE tp_knowledge_ingestion_jobs ENABLE ROW LEVEL SECURITY;

-- Allow authenticated users to read chunks (for now)
CREATE POLICY "Authenticated users can read chunks"
  ON tp_knowledge_chunks FOR SELECT
  TO authenticated USING (true);

-- Users can only see their own query logs
CREATE POLICY "Users see own query logs"
  ON tp_query_logs FOR SELECT
  USING (auth.uid() = user_id);
```

🟠 P1 — Add a Reranker

What's wrong

The current pipeline retrieves the top 10 chunks by cosine similarity, filters anything below 0.8, and sends the rest directly to Claude. This has two problems:

1. The `>= 0.8` threshold is a rough guess — it rejects chunks that might actually be relevant and accepts chunks that look similar but don't answer the question.
2. Vector similarity measures "are these words close in meaning?" not "does this chunk actually answer the question?" A cross-encoder reranker does the latter.

What to do

Add Cohere's `rerank-3.5` model between retrieval and answer generation. The flow becomes:

> Vector search top 30 → Rerank → Take top 8 → Send to Claude

Remove the `>= 0.8` threshold — the reranker replaces it with a learned relevance score.

```typescript
// After retrieving chunks from tp_match_knowledge_chunks
const cohere = new CohereClient({ token: Deno.env.get("COHERE_API_KEY") });

const reranked = await cohere.rerank({
  model: "rerank-3.5",
  query: userQuery,
  documents: chunks.map(c => c.chunk_text),
  topN: 8,
});

const topChunks = reranked.results.map(r => chunks[r.index]);
```

Add `COHERE_API_KEY` to your edge function environment variables. Cost is ~$1 per 1,000 queries. Expected quality improvement: 15–25% on retrieval benchmarks.

🟠 P1 — Add Hybrid Search (Keyword + Vector)

What's wrong

Pure vector search is good at semantic similarity but bad at exact keyword matches. For a trade advisor, this is a real problem — users ask about things like `GLOBALG.A.P`, `HACCP`, `HS code 0902.30`, `EUR-1 certificate`, `phytosanitary`. These terms break into low-quality subwords in embedding models. Keyword (BM25) search handles them perfectly.

What to do

Add a `tsvector` column to `tp_knowledge_chunks` and a GIN index for full-text search. Then run both searches and merge results using Reciprocal Rank Fusion (RRF).

```sql
-- Add the keyword search column (auto-populated)
ALTER TABLE tp_knowledge_chunks
  ADD COLUMN chunk_tsv tsvector
  GENERATED ALWAYS AS (to_tsvector('english', chunk_text)) STORED;

CREATE INDEX tp_chunks_tsv ON tp_knowledge_chunks USING gin(chunk_tsv);
```

Then update `tp_match_knowledge_chunks` to run both searches in parallel and fuse using RRF:

sql

```sql
-- RRF formula: 1 / (rank + 60) summed across retrieval methods
-- Merge vector results and BM25 results, sort by combined RRF score
WITH vector_results AS (
  SELECT id, chunk_text, 1 - (embedding <=> query_embedding::halfvec) AS score,
         ROW_NUMBER() OVER (ORDER BY embedding <=> query_embedding::halfvec) AS rank
  FROM tp_knowledge_chunks
  LIMIT 30
),
bm25_results AS (
  SELECT id, chunk_text, ts_rank_cd(chunk_tsv, query) AS score,
         ROW_NUMBER() OVER (ORDER BY ts_rank_cd(chunk_tsv, query) DESC) AS rank
  FROM tp_knowledge_chunks
  WHERE chunk_tsv @@ query
  LIMIT 30
),
fused AS (
  SELECT COALESCE(v.id, b.id) AS id,
         COALESCE(1.0/(v.rank + 60), 0) + COALESCE(1.0/(b.rank + 60), 0) AS rrf_scor
  FROM vector_results v
  FULL OUTER JOIN bm25_results b ON v.id = b.id
)
SELECT * FROM fused ORDER BY rrf_score DESC LIMIT 30;
```

This pairs with the reranker: hybrid search produces 30 candidates → reranker picks the best 8.

🟠 P1 — Add Query Rewriting for Multi-Turn Conversations

What's wrong

When a user asks a follow-up like "What about Asia?", the system embeds `"What about Asia?"` and searches for that. It finds nothing useful because the context from the previous turns is lost at retrieval time. The history is only passed to Claude for answer *generation* — not for *retrieval*.

What to do

Before embedding the user's query, make a cheap Haiku call to rewrite it as a standalone question using conversation history.

typescript

```
// Add this step before embedding (index.ts around line 362)
async function rewriteQuery(history: Message[], currentMessage: string): Promise<str
  if (history.length === 0) return currentMessage; // No rewriting needed for first

  const response = await anthropic.messages.create({
    model: "claude-haiku-4-5-20251001",
    max_tokens: 150,
    system: "Rewrite the user's latest message as a complete standalone search query
    messages: [
      ...history.slice(-4).map(m => ({ role: m.role, content: m.content })),
      { role: "user", content: `Rewrite this as standalone query: "${currentMessage}
    ]
  });

  return response.content[0].text;
}

// Then embed the rewritten query instead of the raw message
const queryToEmbed = await rewriteQuery(conversationHistory, userMessage);
const embedding = await embedText(queryToEmbed, GEMINI_API_KEY, "RETRIEVAL_QUERY");
```

Cost: ~$0.0001 per turn (Haiku is very cheap). Impact: large improvement on multi-turn conversations.

🟠 P1 — Replace the Fake Progress Bar with Real Token Streaming

What's wrong

The current system buffers the full answer and then artificially drips progress events to the UI at 1.5 second intervals. This makes the app feel slower than it actually is and adds ~80 lines of complex state-management code (`progressQueueRef`, `displayedIndexRef`, `progressIntervalRef`, etc.) that can just be deleted.

Anthropic's API natively supports streaming — Claude sends tokens as it generates them, so the user sees the answer being written in real time (like Claude.ai or ChatGPT).

What to do

In the edge function — switch from `await anthropic.messages.create(...)` to `anthropic.messages.stream(...)`:

```
typescript
```

```
const stream = await anthropic.messages.stream({
  model: "claude-haiku-4-5-20251001",
  max_tokens: 1000,
  system: systemPrompt,
  messages: conversationHistory,
});

// Stream tokens directly to the browser via SSE
const encoder = new TextEncoder();
const readable = new ReadableStream({
  async start(controller) {
    for await (const chunk of stream) {
      if (chunk.type === "content_block_delta" && chunk.delta.type === "text_delta")
        controller.enqueue(encoder.encode(`data: ${JSON.stringify({ token: chunk.del
      }
    }
    controller.enqueue(encoder.encode("data: [DONE]\n\n"));
    controller.close();
  }
});

return new Response(readable, { headers: { "Content-Type": "text/event-stream" } });
```

In `useTradePilotChat.ts` — remove `progressQueueRef`, `displayedIndexRef`, `progressIntervalRef`, `isCompleteRef`, and `displayedProgress`. Replace with a simple token append:

typescript

```
// On each SSE message
if (event.data !== "[DONE]") {
  const { token } = JSON.parse(event.data);
  setMessages(prev => {
    const last = prev[prev.length - 1];
    return [...prev.slice(0, -1), { ...last, content: last.content + token }];
  });
}
```

Net result: better UX, ~150 fewer lines of code.

🟠 P1 — Build a Basic Eval Suite

What's wrong

There are currently zero tests on the RAG pipeline. Every prompt change, model change, or

chunking change is a guess — you have no way to know if you helped or hurt retrieval quality. This is the main difference between a demo and a production product.

What to do

Retrieval evals (highest priority): Create a file `evals/retrieval.json` with 30–50 hand-curated pairs:

```json
[
  {
    "question": "What documents are needed to export agricultural products to German
    "ideal_chunk_ids": ["chunk_abc123", "chunk_def456"]
  },
  ...
]
```

Write a script that embeds each question, runs retrieval, and checks whether the ideal chunks appear in the top 5 and top 10 results. Track `recall@5` and `recall@10`. Run this after any change to the embedding model, chunker, or retrieval settings.

Answer evals: After retrieval evals are passing, add 30 question/expected-answer pairs and use Claude Sonnet as a judge to score each answer for groundedness and completeness.

Use `tp_query_logs` as a source — the query logs already capture questions, retrieved chunks, and answers. Mine it for real questions to build the eval set.

🟡 P2 — Batch Embeddings in Ingest

What's wrong

`trade-pilot-ingest/index.ts` embeds chunks one at a time in a `for` loop:

```typescript
for (const chunk of chunks) {
  embedding = await embedText(chunk.chunk_text, ...); // one round-trip per chunk
}
```

A 50-chunk article = 50 sequential Gemini API calls ≈ 10 seconds of blocking. The request just hangs. The `tp_knowledge_ingestion_jobs` table was clearly designed to support async queue-based processing but it's not being used that way.

What to do (quick fix)

Use Gemini's `batchEmbedContents` endpoint to embed all chunks in 1–2 calls:

```typescript
// Instead of the for loop
const batchResponse = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/text-embedding-004:batchE
  {
    method: "POST",
    body: JSON.stringify({
      requests: chunks.map(chunk => ({
        model: "models/text-embedding-004",
        content: { parts: [{ text: chunk.chunk_text }] },
        taskType: "RETRIEVAL_DOCUMENT"
      }))
    })
  }
);
const { embeddings } = await batchResponse.json();
// embeddings[i].values corresponds to chunks[i]
```

This reduces a 50-chunk ingest from ~10 seconds to ~300ms.

🟡 P2 — Replace the Validator with Retrieval Score Checks

What's wrong

`validateAnswer()` asks Haiku to judge an answer that Haiku just generated. The same model, same training, same blind spots — it rubber-stamps its own work ~95% of the time. The validator adds latency and cost without meaningfully improving answer quality.

What to do

Delete `validateAnswer()`. Instead, use the retrieval metrics you already have to decide whether to fall back to web search:

```typescript
const avgSimilarity = chunks.reduce((sum, c) => sum + c.similarity, 0) / chunks.leng
const usefulChunkCount = chunks.filter(c => c.similarity >= 0.75).length;

// Fall back to web search if retrieval is weak
const shouldUseWebSearch = avgSimilarity < 0.72 || usefulChunkCount < 2 || intent ==
```

This is more reliable than asking the model to self-assess, and it removes a full LLM call from the hot path.

🟡 P2 — Replace Citation Regex with Structured Output

What's wrong

`index.ts:498-522` uses regex to parse `[1]`, `[2]` markers out of Claude's response, renumber them, and rebuild the string. This breaks whenever Claude formats citations differently — e.g. `[1, 2]` or `[1][2]` or `[1-3]`.

What to do

Use Claude's tool/function calling to get structured citation output instead of parsing free text:

typescript

```
const tools = [{
  name: "provide_answer",
  description: "Provide the answer with structured citations",
  input_schema: {
    type: "object",
    properties: {
      answer: { type: "string", description: "The answer text with [N] citation mark
      citations: {
        type: "array",
        items: {
          type: "object",
          properties: {
            n: { type: "number" },
            source_id: { type: "string" },
            chunk_id: { type: "string" }
          }
        }
      }
    }
  }
}];

// Then use tool_choice: { type: "tool", name: "provide_answer" }
// The response will be structured JSON — no regex needed
```

What's Already Good — Don't Change These

- `tp_query_logs` schema captures latency, confidence, citations, model name — this is the foundation for evals and observability. Use it.
- `tp_knowledge_ingestion_jobs` table is already designed for queue-based processing. The quick fix above uses batch calls; the proper fix is to wire this as an actual queue.

- Trusted domain prioritisation in web search is a good production pattern. Keep it and expand the list.
- Vancouver-style inline citations are better than footnotes. Keep the approach, just improve the implementation.
- Use-case tagging schema on sources and chunks is already there — the metadata exists but `filter_use_case` is never passed in the chat RPC call. Wiring that up takes ~10 lines and gives metadata-aware retrieval for free.

Dead Code to Delete

Three sections of code do nothing useful and should be removed:

1. `filter_use_case`, `filter_country`, `filter_market` parameters in `tp_match_knowledge_chunks` — never passed by the chat function. Either wire them up or remove them.
2. `classifyQuery()` regex router — 23 lines of regex whose only real effect is toggling web search on/off. Replace with the retrieval score check described above.
3. Citation renumbering block (`index.ts:498-522`) — replaced by structured output.

That's ~80 lines of code that can be removed cleanly.